P8 — The source code: what computes what, and how to compile and run it

Provenance required by Hard Rule 6 of the task specification (PROJECT spacetimes)

Prepared for Patrick Lee Nash, Ph.D.

November 12, 2026

1. What this document is for

Hard Rule 6 of the task specification demands commented source code for every equation, result, figure, graph and prediction, instructions for using the code, a provenance for every computed value, and that the code be executed. This document is the map: it says which program computes which file, which file feeds which figure and which number of the documents, how the programs are tested, and how to compile and run everything.

The rules that the code follows:

- **Rust for every computation.** The task specification prefers Rust. All physics is computed by one Rust program, `spacetimes` (`code/spacetimes-rs`, 28 source files, about 11 400 lines, 63 unit tests). It uses no external library, so it builds with nothing but a Rust compiler.
- **Python for an independent verification, and for tables, figures and documents.** A program can be wrong in the same way twice; the Dirac matrices are therefore verified a second time by a Python program that shares no code with the Rust program. Python scripts also turn the data files into tables and figures and build the documents.
- **Mathematica is not used.** The matrices are read from what Mathematica printed in the author's notebooks (provenance P3) and rebuilt by the programs.
- **No number is typed by hand.** Every number in a document is inserted by a script from a data file that a program wrote (Sec. 6).

Where the code was executed. All data files, figures and documents of the repository were produced by running the code on a Linux computer (x86-64) with rustc 1.97.0, Python 3.13.18 (numpy 2.5.3, scipy 1.18.1, matplotlib 3.11.2), pandoc 3.1.3 and XeLaTeX (TeX Live 2023). The assistant could not run the Rust program on the author's Mac, because no Rust compiler is reachable from its sandbox there; since the program has no dependencies, it should build and run on macOS unchanged (Sec. 2). Results of floating-point arithmetic may then differ in the last digits; the exact checks of the matrices use integer arithmetic and must agree exactly.

2. How to compile and run everything on a Mac

Each block can be pasted into the Terminal as it is. Lines that begin with # are explanations.

Step 1, once: the Rust compiler. If the command `cargo --version` prints a version number, skip this step.

```
# install the Rust compiler and its build tool "cargo" (official installer; accept the defaults)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
# make the new commands known to this Terminal window
source "$HOME/.cargo/env"
cargo --version
```

Step 2, once: the Python packages (needed for the tables, the figures and the comparison with the reference spectra, not for the Rust program).

```
python3 -m pip install --user numpy scipy matplotlib
```

Step 3: compile and test. This compiles the program (about one minute) and runs its 63 unit tests; the last line must say `test result: ok. 63 passed`.

```
cd "/Users/nsh/Developer/github/spacetimes/code/spacetimes-rs"
cargo build --release
cargo test --release
```

Step 4: recompute every data file. Each command writes its files into `data/` and prints what it did. The commands `forecast` (about one hour) and `superfluid` (about ten minutes) are the long ones; the whole block takes about an hour and a half on a computer with two processor cores.

```
cd "/Users/nsh/Developer/github/spacetimes"
B=code/spacetimes-rs/target/release/spacetimes
for c in matrices channels gap dimer symmetry cjt background spectrum forecast superfluid halos
do
  $B $c
done
```

Step 5: the independent verification and the comparison with CLASS.

```
cd "/Users/nsh/Developer/github/spacetimes"
python3 code/scripts/verify_matrices_independent.py
python3 code/scripts/validate_cmb_against_class.py
python3 code/scripts/check_symbols.py
```

Step 6: tables and figures.

```
cd "/Users/nsh/Developer/github/spacetimes"
python3 code/scripts/make_tables.py
python3 code/scripts/make_figures.py
```

Step 7, optional: the documents. This needs pandoc and a TeX installation with XeLaTeX and latexmk (for example MacTeX), and the DejaVu and Latin Modern Math fonts.

```
cd "/Users/nsh/Developer/github/spacetimes"
for f in docs_src/*.src.md; do python3 code/scripts/assemble_docs.py "$f"; done
bash code/scripts/build_docs.sh
python3 code/scripts/build_manuscripts.py
```

To run a single command of the program and see the list of all commands:

```
cd "/Users/nsh/Developer/github/spacetimes"
code/spacetimes-rs/target/release/spacetimes help
```

3. The commands of the Rust program, and the files they write

Every data file begins with lines that start with # and state the program, the command, the content, the units and the parameters; the table can be checked against these headers.

Command	What it computes	Files in `data/`	Documented in
`matrices`	builds the two sets of eight real 16×16 Dirac matrices and runs the 106 machine checks; classifies the fermion pairs	`gamma1644.txt`, `gamma1671.txt`, `matrix_checks.txt`, `matrix_checks.csv`, `pair_classification.txt`, `pair_classification.csv`; also `provenance/generated/matrices.md`	“dirac matrices”; M II to IV
`channels`	exchange amplitudes of the Vorton-Scalar in the pair channels; quartic invariant of the order parameter	`pair_channel_kernel.csv`, `gl_quartic_check.csv`	S1 Secs. 4, 6; M VI, VIII

Command	What it computes	Files in data/	Documented in
gap	gap equation of the hidden-axis singlet	gap_scan.csv, gap_profile.csv	S1 Sec. 5; M VII
dimer	two fermions in the Yukawa potential: scattering length, binding energy, critical coupling	dimer.csv	S1 Sec. 5.3; M VII
symmetry	internal symmetry of the zero-mode sector, from the matrices	reduced_symmetry.csv	S1 Sec. 2.1; S4 Sec. 2.4; M V
cjt	everything computed with the effective action of Cornwall, Jackiw and Tomboulis: test model, paired state, thermodynamics, order-parameter landscape, matter, temperature, vacuum	the sixteen files cjt_*.csv	S4; M VII to IX, App. B
background	expansion history and recombination with the dark radiation of the model	step_thermo.csv	S3 Sec. 1; M XI
spectrum	spectra of the microwave background and of matter for the fiducial model; convergence	cmb_fiducial_cl.csv, cmb_fiducial_pk.csv, cmb_convergence.csv	S3 Sec. 2; M XI, App. D
forecast	fits of seven families of dark-radiation models to mock data	dr_fisher.csv, dr_profiles.csv, dr_residuals.csv, dr_zt_scan.csv, dr_chord_check.csv	S3 Sec. 3; M XI
superfluid	growth of structure if the dark matter is a superfluid	sfdm_transfer.csv, sfdm_summary.csv, sfdm_bounds.csv	S3 Sec. 4; M XII
halos	conditions for degeneracy and condensation in halos; self-interaction of bound pairs	dm_halo_estimates.csv, dm_selfinteraction.csv	S3 Sec. 4; M XII

Files in data/ that other programs write: matrix_checks_independent.txt (verify_matrices_independent.py); cmb_validation.csv, pk_validation.csv (validate_cmb_against_class.py); notebook_matrices_4+4.txt, notebook_matrices_7+1.txt (tools/extract_nb_matrices.py, provenance P3); symbol_checks.txt, symbol_concordance.csv (provenance P1); and the two reference spectra in data/reference/, which are copies from the author's project NCSS with their checksums in the headers.

4. The source files of the Rust program

Each file begins with a comment that states its purpose, its equations and its tests. Four small files are taken unchanged from the author's project NCSS and say so in their first lines.

File	Lines	Purpose
gamma.rs	270	construction of the two sets of Dirac matrices from their definitions
imat.rs	316	exact matrix arithmetic over the integers and the Gaussian integers; exact rank
verify.rs	582	the machine checks of the matrices (anticommutators, Lie algebra, group, notebooks, chirality)
dirac.rs	272	the Dirac equations of the two fields, checked in momentum space
pairs.rs	556	which pairs of two identical particles exist; their quantum numbers
symmetry.rs	551	internal symmetries of the zero-mode sector
kernel.rs	463	exchange amplitudes of the Vorton-Scalar, channel by channel
gap.rs	414	the gap equation of the hidden-axis singlet
dimer.rs	171	two particles in the Yukawa potential
cjt_toy.rs	201	the method of Cornwall, Jackiw and Tomboulis on a solvable model
cjt_pairing.rs	778	effective potential of the paired state; thermodynamics; landscape
cjt_normal.rs	807	effective potential for the expectation value of the Vorton-Scalar: matter, temperature
cjt_vacuum.rs	163	the gap equation in the vacuum

File	Lines	Purpose
`cjt_cmd.rs`	612	the command `cjt`
`cosmo.rs`	1108	expansion history, the dark radiation of the model, recombination
`boltzmann.rs`	727	linear perturbations: the Einstein-Boltzmann equations
`spectra.rs`	358	angular power spectra and matter power spectrum
`bessel.rs`	222	spherical Bessel functions for the line-of-sight integrals
`forecast.rs`	535	mock likelihood, Fisher matrices, profile fits
`fmat.rs`	187	small real matrices in double precision
`quad.rs`	146	Gauss-Legendre quadrature
`export.rs`	136	writing matrices and check reports
`main.rs`	978	the commands
`lib.rs`	56	list of the modules
`complex.rs`, `linalg.rs`, `roots.rs`, `io.rs`	745	complex numbers, small Hermitian eigenproblems, root finding, CSV output (from NCSS)

5. The Python scripts and tools

Script	Purpose
`code/scripts/verify_matrices_independent.py`	independent verification of the matrices: rebuilds them from the definitions with exact rational arithmetic and repeats the essential checks (36 checks); writes `data/matrix_checks_independent.txt`
`code/scripts/validate_cmb_against_class.py`	compares the spectra of the Rust program with reference spectra of the code CLASS; writes `cmb_validation.csv`, `pk_validation.csv`
`code/scripts/make_tables.py`, `make_tables_cosmo.py`, `make_tables_cjt.py`	turn the data files into the numbers and tables of the documents (`provenance/generated/tables.md`)
`code/scripts/make_figures.py`	draws all figures from the data files (Sec. 7)
`code/scripts/assemble_docs.py`	inserts the generated numbers, tables and files into the document sources `docs_src/*.src.md`
`code/scripts/build_docs.sh`, `check_layout.py`, `doc_header.tex`	Markdown to LaTeX to PDF for the supplements and provenances, with a check of the layout
`code/scripts/build_manuscripts.py`, `tex2md.py`	build the two manuscripts from the common REVTeX source in `docs_src/manuscript/`; Markdown versions
`code/scripts/write_symbol_registry.py`, `check_symbols.py`, `symbol_concordance.py`	the registry of symbols and its checks (provenance P1)
`tools/nb2txt.py`, `tools/extract_nb_matrices.py`, `tools/scan_notebook_symbols.py`	reading the notebooks without Mathematica (provenance P3)

6. From a data file to a number in a document

No number in the supplements, in “dirac matrices” or in the manuscripts is typed by hand. The chain is

1. a command of the Rust program writes a data file (Sec. 3);
2. `make_tables.py` reads the data files and writes named fragments (a number, a table) into `provenance/generated/tables.md`; `spacetimes matrices` writes the displayed matrices into `provenance/generated/matrices.md`;
3. the document sources contain placeholders (the name of a fragment in double curly braces, preceded by `GEN:`, `GENM:` or `ROWS:`); `assemble_docs.py` and `build_manuscripts.py` replace them.

To find the origin of a number, search for it in `provenance/generated/tables.md`; the name of the fragment leads to the function of `make_tables*.py` that made it, and that function names the data

file and the column. To change a number, the program must be changed and run again; editing a document does not survive the next build.

7. From a data file to a figure

All figures are drawn by `code/scripts/make_figures.py`, one function per figure; each figure exists as PDF (for the documents) and PNG.

Figure (`figures/`)	Data files	Shows
`fig_kernels`	`pair_channel_kernel.csv`	exchange amplitudes in the pair channels
`fig_gap`	`gap_scan.csv`	the gap and the critical temperature
`fig_dimer`	`dimer.csv`	scattering length and binding energy of two fermions
`fig_cjt_toy`	`cjt_toy.csv`	the effective action on a solvable model
`fig_cjt_thermo`	`cjt_thermo.csv`	gap, entropy and heat capacity against temperature
`fig_cjt_landscape`	`cjt_landscape_scan.csv`	energy as a function of the order parameter
`fig_cjt_normal`	`cjt_normal.csv`	fermion mass in matter; self-bound matter
`fig_cjt_thermal`	`cjt_thermal_profile.csv`	the fermion mass at finite temperature
`fig_cjt_vacuum`	`cjt_vacuum.csv`	the critical coupling for a condensate in the vacuum
`fig_cjt_vacpot`	`cjt_vacuum_potential.csv`	the potential of the Vorton-Scalar in the vacuum
`fig_cmb_validation`	`cmb_validation.csv`	spectra of this work against CLASS
`fig_step_thermo`	`step_thermo.csv`	the dark radiation of the model through its transition
`fig_dr_profiles_planck`, `fig_dr_profiles_deep`	`dr_profiles.csv`	fits of the dark-radiation families to the two mock data sets
`fig_dr_residuals`	`dr_residuals.csv`	residual spectra
`fig_dr_zt`	`dr_zt_scan.csv`	dependence on the transition redshift
`fig_sfdm_transfer`	`sfdm_transfer.csv`, `sfdm_summary.csv`, `sfdm_bounds.csv`	suppression of the matter power spectrum by the sound of a superfluid
`fig_dm_windows`	`dm_halo_estimates.csv`, `dm_selfinteraction.csv`, `sfdm_bounds.csv`	the fermion masses that a galactic superfluid needs and those that observations allow

8. How the code is tested

Test	What it establishes	Result	Where
106 machine checks of `spacetimes matrices` (96 exact, 10 numerical)	the properties of the two sets of Dirac matrices, including identity with the matrices printed in the notebooks	all pass	`data/matrix_checks.txt`
36 checks of the independent Python program	the same essential properties by other methods	all pass	`data/matrix_checks_independent.txt`
63 unit tests (`cargo test --release`)	each numerical routine against a closed result or an independent method	all pass	the `tests` modules of the source files
comparison with CLASS	the Einstein-Boltzmann solver against reference spectra	differences in the tables of S3 Sec. 2 and of M App. D	`data/cmb_validation.csv`, `data/pk_validation.csv`

Test	What it establishes	Result	Where
consistency checks through the effective action	the gap equation, the bound-state threshold and the Ginzburg-Landau coefficients by a second route	agreement stated in S4 Sec. 3	`data/cjt_pairing_check.csv`, `cjt_threshold.csv`, `cjt_gl_summary.csv`
9 checks of the symbols	uniqueness of the symbols of the project	all pass	`data/symbol_checks.txt`
layout checks	no line, table or formula wider than its box; no undefined reference	reported at every build	output of `build_docs.sh`, `build_manuscripts.py`

The code CLASS itself was not run in this project: the reference spectra are those of the author's project NCSS, computed there with CLASS v3.4.2. Every error that the tests found is recorded in the error log (`provenance/P9_error_log`).